Comparing Two String Objects in Java

In Java, there are several ways to compare two `String` objects. The two most commonly used methods for comparing strings are `compareTo()` and `equals()`.

1. `compareTo()` Method

The `compareTo()` method compares two strings lexicographically (based on the Unicode value of each character in the strings).

- **Return Value**:
  - **0**: if the strings are equal.
  - **A negative integer**: if the first string is lexicographically less than the second string.
  - **A positive integer**: if the first string is lexicographically greater than the second string.

Syntax:

```
int compareTo(String anotherString)
```

Example:

```
public class StringCompareToExample {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "hello";
        String str3 = "Hello";

        // Comparing str1 and str2
        int result1 = str1.compareTo(str2);
        System.out.println("Result of compareTo(str1, str2): " + result1);
// Should print a negative value (since 'H' < 'h')

        // Comparing str1 and str3
        int result2 = str1.compareTo(str3);
        System.out.println("Result of compareTo(str1, str3): " + result2);
// Should print 0 (since they are equal)

        // Comparing str2 and str3
        int result3 = str2.compareTo(str3);
        System.out.println("Result of compareTo(str2, str3): " + result3);
// Should print a positive value (since 'h' > 'H')
```

```
    }
}
```

Output:

```
Result of compareTo(str1, str2): -32
Result of compareTo(str1, str3): 0
Result of compareTo(str2, str3): 32
```

Explanation:

- The difference between 'H' and 'h' in Unicode is **32**, which is why the result of `str1.compareTo(str2)` is **-32**, indicating that "Hello" is lexicographically less than "hello".
- When comparing identical strings like `str1.compareTo(str3)`, the result is **0**, indicating that both strings are equal.

2. `equals()` Method

The `equals()` method compares two strings for **exact equality** (not lexicographical order). It checks whether the two strings have the same characters in the same order.

- **Return Value**:
  - `true` if the strings are exactly equal (case-sensitive).
  - `false` if the strings are not equal.

Syntax:

```
boolean equals(Object anObject)
```

Example:

```
public class StringEqualsExample {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "hello";
        String str3 = "Hello";

        // Comparing str1 and str2
        boolean result1 = str1.equals(str2);
        System.out.println("Result of equals(str1, str2): " + result1); //
Should print false (case-sensitive comparison)

        // Comparing str1 and str3
        boolean result2 = str1.equals(str3);
```

```
        System.out.println("Result of equals(str1, str3): " + result2); //
Should print true (both are exactly the same)
    }
}
```

Output:

```
Result of equals(str1, str2): false
Result of equals(str1, str3): true
```

Explanation:

- Since `equals()` is case-sensitive, `str1.equals(str2)` returns **false** because "Hello" is not equal to "hello" due to the difference in case.
- `str1.equals(str3)` returns **true** because both strings are exactly the same.

Key Differences Between `compareTo()` and `equals()`

Feature	`compareTo()`	`equals()`
Purpose	Compares two strings lexicographically (in dictionary order).	Checks if two strings are exactly equal (case-sensitive).
Return Value	- 0 if strings are equal. - Negative value if the first string is lexicographically less. - Positive value if the first string is lexicographically greater.	`true` if strings are exactly equal. `false` if not equal.
Case Sensitivity	Yes, it is case-sensitive.	Yes, it is case-sensitive.
Use Case	To compare strings and determine their relative order.	To check if two strings are exactly the same.